

---

# Comparison of Message Passing Graph Neural Networks and the Weisfeiler Lehman Graph Isomorphism Test

---

Ryan Himes  
University of Pennsylvania  
ryhime@seas.upenn.edu

## Abstract

1        Message passing graph neural networks (MP-GNNs) can model structured graph  
2        data such as citations, computer networks, and social media connections. While  
3        being able to run on more general structures that other neural networks cannot  
4        capture completely, MP-GNNs have many limitations. One of the main theoretical  
5        limitations of MP-GNNs is that they are restricted by the Weisfeiler Lehman (WL)  
6        graph isomorphism test. The WL test is known to produce false positives while  
7        predicting if there exists an isomorphism between two graphs, which poses an  
8        expressivity barrier to MP-GNNs. This paper explores the relationship between  
9        the WL test, changes to MP-GNNs to be as powerful as the WL test, and proposed  
10        ways to overcome the theoretical barrier of the WL test. Additionally, experiments  
11        will be done along the way to support theoretical findings and pose questions.

## 12    1    Introduction

13    Message passing graph neural networks (MP-GNN) can take many forms depending on which  
14    functions are used within their message passing structure. Finding the optimal structure for MP-  
15    GNNs is crucial to getting the most out of graph structured data. To determine the best structure  
16    that a MP-GNN can take, theoretical findings comparing MP-GNN to the Weisfeiler Lehman (WL)  
17    graph isomorphism test has been done. It has been shown that classic MP-GNNs can be at most as  
18    expressive as the WL test and as expressive when certain conditions of the functions used are met.

19    This paper explores the reasoning behind why classic MP-GNNs can be at most as expressive  
20    as the WL test as well as modifications that have been made in attempt to overcome that barrier.  
21    After the analysis of some papers, home experiments are done to backup theoretical claims such as  
22    experimentally comparing theoretically backed up variations of MP-GNNs to the WL test.

23    Since detecting isomorphisms with the WL test are covered extensively throughout, an isomorphism  
24    between two graphs,  $G$  and  $G'$  is defined here.

25    An isomorphism exists between two graphs  $G = (V, E)$  and  $G' = (V', E')$  if there exists a bijection  
26     $f : V \rightarrow V'$  where  $\{v, u\} \in E \iff \{f(v), f(u)\} \in E'$ . This means an isomorphism between two  
27    graphs exists if there exists a one-to-one mapping between their vertices such that two vertices are  
28    connected by an edge in one graph,  $G$ , if and only if their mapped vertices are connected by an edge  
29    in the other graph,  $G'$ . This is important to GNNs since the ability to distinguish that graph structure  
30    is different to classify two graphs into different classes for graph level classification. GNNs also have  
31    an inherent advantage over the WL isomorphism test while training on datasets, since GNNs can  
32    learn trainable weights to adapt to the distribution of the dataset,  $\mathcal{D}$ , while the WL test is a set test for  
33    all graphs but cannot adapt any type of weights to a specific distribution.

## 34 2 Message Passing Graph Neural Networks

35 MP-GNNs is a subset of GNNs which pass messages from node to node using aggregate and  
 36 update steps for each iteration. Formally, a GNN takes as input a directed or undirected graph  
 37  $G = (V, E)$ , where  $V$  is the set of vertices and  $E$  is the set of edges where  $E \subseteq V \times V$ . Each vertex,  
 38  $v \in V$ , is given an initial feature vector,  $h_v^{(0)} \in \mathbb{R}^{d_v^0}$ , and each edge,  $e$ , is given an initial feature  
 39 vector  $e_{uv} \in \mathbb{R}^{d_e^0}$ .  $d_v^0$  represents the dimension of the initial feature vector for each vertex while  $d_e^0$   
 40 represents the dimension of the initial feature vector for each edge. Each of the initial feature vectors  
 41 are used to represent the meaning of each edge and vertex. In the case of social networks, a vertex  
 42 feature vector could represent the attributes of a person, while the edge feature vector could represent  
 43 the relationship between two people. A GNN layer,  $f_k$ , maps each vertex feature vector to a new  
 44 feature vector,  $h_v^k \in \mathbb{R}^{d_v^k}$ .

45  
 46 A MP-GNN uses the functions *MESSAGE*, *AGGREGATE*, *UPDATE* for the forward  
 47 pass of each layer. Changing these two functions are primary ways of creating different MP-GNNs.  
 48 The value for vertex value,  $h_v^{(k)}$ , at MP-GNN layer  $k$  is shown below in Equations 1 and 2. Equation  
 49 1 aggregates the information from neighboring nodes first applying the *MESSAGE* function, to  
 50 each which uses the two nodes,  $h_u^{(k-1)}$  and  $h_v^{(k-1)}$ , and the edge feature between the two,  $e_{uv}$ . Then  
 51 the aggregated information,  $m_v^{(k)}$ , is used with the feature of the same node from the last layer,  
 52  $h_v^{(k-1)}$ , to update the node's feature,  $h_v^{(k)}$ .

$$m_v^{(k)} = \text{AGGREGATE}^{(k)}(\{\text{MESSAGE}(h_u^{(k-1)}, h_v^{(k-1)}, e_{uv}) : u \in \mathcal{N}(v)\}) \quad (1)$$

$$h_v^{(k)} = \text{UPDATE}(h_v^{(k-1)}, m_v^{(k)}) \quad (2)$$

53 This is applied to every vertex in the graph to produce a graph with new vertex features. For some  
 54 problems the graph needs to be mapped to a single vector representation for tasks like graph level  
 55 classification and regression. To do this a *READOUT* function is defined that maps a graph to a  
 56 vector which is defined in Equation 3. Readout functions can take a variety of different forms such  
 57 as adding all the values in each feature vector and using that for an index in the output vector. In  
 58 this example, the readout function would map to a vector  $y \in \mathbb{R}^{|V|}$ , but others can map to different  
 59 dimensions such as using the top  $n$  entries in each vector would map to  $y \in \mathbb{R}^{n|V|}$ .

$$y = \text{READOUT}(\{h_v^{(K)} : v \in V\}) \quad (3)$$

60 It is widely accepted that the *AGGREGATE* and *READOUT* functions should be permutation-  
 61 invariant. Permutation-invariance is defined as changing the order of the input does not change the  
 62 output. Some examples of this are summation, mean, and average, while concatenation is an example  
 63 of an operation that is not permutation-invariant. This is done since the order the vertices are given  
 64 to the MP-GNN should not affect graph structure. Formally a function  $f$  is permutation invariant if  
 65 after applying a shift,  $\pi$ ,  $f(x) = f(\pi(x))$  for all  $x$  values.

## 66 3 Weisfeiler Lehman Graph Isomorphism Test

67 This section outlines the Weisfeiler Lehman Graph Isomorphism Test as well as a simple example of  
 68 determining that there does not exist an isomorphism between two graphs. To run the WL test on two  
 69 different graphs to determine if there exists an isomorphism between the two, graphs are initialized  
 70 with the same color assigned to each vertex. For every iteration, colors are updated, then the multiset  
 71 of colors assigned to each vertex is compared, if the multiset of colors of the two graphs are different  
 72 after any iteration there is guaranteed to be no isomorphism between the two graphs. The iterations  
 73 continue until the algorithm converges by having the same pattern of colors assigned throughout the  
 74 graph after many iterations. This means that if for every set of vertices that are colored the same still  
 75 share a common color, after many iterations, the algorithm converges. If the multisets continue to be  
 76 identical after each iteration, there is no way to guarantee that there exists an isomorphism between  
 77 the two graphs since the WL test can only reliably tell if there is no isomorphism between two graphs.

78 First, the definition of a multiset is a 2-tuple  $X = (S, m)$  where  $S$  is the underlying set of  $X$  that is  
 79 formed from its distinct elements, and  $m : S \rightarrow \mathbb{N}_{\geq 1}$  gives the multiplicity of the elements. (1)

80 During each iteration of the WL test, the color assigned to each vertex can be updated according to  
 81 Equation 4. A requirement is that the *HASH* function must be injective so multisets that differ in  
 82 values do not map to the same hash value. This is because  $\{c_u^{(t)} : u \in \mathcal{N}(v)\}$  is a multiset of color  
 83 hashes from neighboring vertices.

$$c_v^{(t+1)} = \text{HASH}(c_v^{(t)}, \{c_u^{(t)} : u \in \mathcal{N}(v)\}) \quad (4)$$

84 An example of this test is shown below in Figure 1. The multiset for  $G_1$  af-  
 85 ter one iteration is  $\{\text{Red}, \text{Red}, \text{Red}, \text{Red}, \text{Yellow}, \text{Green}\}$ , while  $G_2$  has the multiset  
 86  $\{\text{Red}, \text{Red}, \text{Red}, \text{Green}, \text{Green}, \text{Yellow}\}$  which do not equal each other so after one iteration  
 87 the WL test determines there is no isomorphism between  $G_1$  and  $G_2$ .

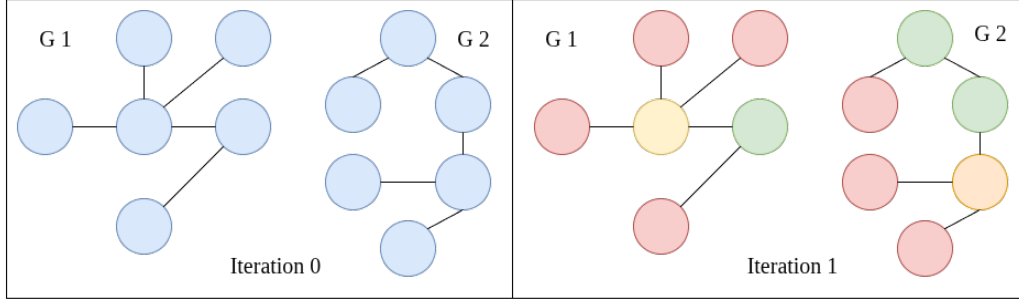


Figure 1: An Example of Running the Weisfeiler Lehman Test

## 88 4 Graph Isomorphism Networks

89 The paper that introduces the graph isomorphism network starts by showing that MP-GNNs are at  
 90 most as powerful as the WL test in **Lemma 2** then shows in **Theorem 3** that with certain modifications  
 91 to a MP-GNN, it can be equal in expressive power.

92 **Lemma 2:** Let  $G_1$  and  $G_2$  be any two non-isomorphic graphs. If a graph neural network  $\mathcal{A} : \mathcal{G} \rightarrow \mathbb{R}^d$   
 93 maps  $G_1$  and  $G_2$  to different embeddings, the Weisfeiler-Lehman graph isomorphism test also decides  
 94  $G_1$  and  $G_2$  are not isomorphic.

95 The idea of this lemma is to show that classic MP-GNNs can be at most as powerful as the WL test.  
 96 The full proof for the lemma is in the original paper (1). The authors do a proof by contradiction  
 97 that the GNN,  $\mathcal{A}$  has  $\mathcal{A}(G_1) \neq \mathcal{A}(G_2)$  but the WL test cannot decide  $G_1$  and  $G_2$  have the same  
 98 collection of node colors. This is done with induction by showing there exists a mapping from the  
 99 multiset produced after each iteration of the WL test to the corresponding feature vectors of each  
 100 vertex produced after each GNN layer. This creates a contradiction to prove their lemma.

101 **Theorem 3:** Let  $\mathcal{A} : \mathcal{G} \rightarrow \mathbb{R}^d$  be a GNN. With sufficient number of GNN layers,  $\mathcal{A}$  maps any graphs  
 102  $G_1$  and  $G_2$  that the Weisfeiler Lehman test of isomorphism decides as non-isomorphic, to different  
 103 embeddings if the following conditions hold:

104 a.)  $\mathcal{A}$  aggregates and updates node features iteratively with  $h_v^{(k)} = \psi(h_v^{(k-1)}, f(\{h_u^{(k-1)} : u \in$   
 105  $\mathcal{N}(v)\}))$  where the functions  $f$ , which operates on multisets, and  $\psi$  is injective.

106 b.)  $\mathcal{A}$ 's graph-level readout, which operates on the multiset of node features  $\{h_v^{(k)}\}$  is injective.

107 An injective function is a function that maps each element in the domain to a unique element in the  
 108 co-domain, or more formally  $f : X \rightarrow Y$  is injective if and only if  $f(x) = f(y) \Rightarrow x = y$ .

109 The full proof for Theorem 3 can be found in the original graph isomorphism network paper (1). The  
 110 proof takes an inductive approach showing that after each layer, there exists an injective function  $\varphi$   
 111 where  $h_v^{(k)} = \varphi(l_v^{(k)})$ , where  $h_v^{(k)}$  is the output of the  $k$ th layer of the GNN and  $l_v^{(k)}$  is the output of

the  $k$ th iteration of the WL test. By showing that  $\varphi$  exists, if the WL test produces differing multisets, the injective function  $\varphi$  maps the differing multisets to differing embeddings produced by the GNN, due to the injectivity of  $\varphi$ . This is related to  $\psi$  being injective following the same logic.  $f$  must work on multisets since the WL test can produce multisets while aggregating colors of neighboring vertices. For (b), the *READOUT* function must also be injective following the same logic that  $\varphi$  must be injective, if the *READOUT* function is not injective it has the ability to map two graphs that differ to the same point which should be avoided. This sets the theoretical upper bound of MP-GNNs to the WL test if they follow requirements (a) and (b).

Since **Lemma 2** shows the lower bound and **Theorem 3** shows an upper bound of expressiveness for MP-GNNs compared to the WL test, if the MP-GNN follows (a) and (b) it will be as expressive as the WL test.

To satisfy requirements (a) and (b),  $\psi = MLP$  which is an injective function and  $f = \sum$  which works on multisets. They do not define a *READOUT* function, so (b) is automatically satisfied.  $(1 + \epsilon^{(k)})$  is a vector that can be learned to assign weight to the node's own feature while using neighboring node values in the update process.

$$h_v^{(k)} = MLP^{(k)}((1 + \epsilon^{(k)}) \cdot h_v^{(k-1)} + \sum_{u \in \mathcal{N}(v)} h_u^{(k-1)}) \quad (5)$$

The authors then test their GIN implementation on different datasets for different classification tasks. GIN was able to outperform the WL subtree kernel in all datasets. The WL subtree kernel is a way to modify the WL test to a classification task using a support vector machine by using the features/colors extracted by the WL test as input to a classifier such as a support vector machine. Even though the WL test and GIN should be as expressive as each other, GIN outperforms the WL test since GNNs are able to learn on the specific distribution of the dataset,  $\mathcal{D}$ , while the WL test, which produces the graph encoding used by the support vector machine, does not have weights to learn to the specific dataset distribution directly from the graph structure.

This implementation has some positives as well as negatives. The main positive is that the GIN implementation is as scalable as other MP-GNNs since the only requirements of GIN is that  $\psi$  is injective,  $f$  works on multisets, and *READOUT* is injective. This allows the message passing in GIN to be run in parallel for each vertex  $v \in V$ . The major downside of GIN is the theoretical side, that it is only as powerful as the WL test. This puts a strict upper bound of how expressive GIN is, so to create a GNN architecture that is more expressive than GIN the new architecture must be more expressive than the WL test.

#### 4.1 Home Experimentation

To show in an experimental setup that when the WL test says there is no isomorphism between two graphs implies a GNN maps the two graphs to different embeddings if and only if requirements (a) and (b) are met, a isomorphism test is setup with different GNN architectures. The test generates 1000 random graphs that do not have a isomorphism between them, then runs a GNN and the WL test. The WL test should output that there is no isomorphism while the GNN should output different embeddings if and only if (a) and (b) are satisfied. The first test is done with a similar structure to the original GIN architecture setting  $\psi = MLP$ , which is injective, and  $f = \sum$  which works on multisets. This satisfies requirement (a). To satisfy requirement (b), *READOUT* is set to be concatenation which is known to be injective. This is shown in Equation 6. The *MLP* is set to have random weights since in this case it just needs to be an injective function.

$$h_v^{(k)} = MLP(h_v^{(k-1)}, \sum(\{h_u^{(k-1)} : u \in \mathcal{N}(v)\})) \quad (6)$$

Using this GNN, when the WL test determines there is no isomorphism the GNN maps the two graphs to different embeddings for all 1000 graphs.

To show that when one of the requirements is broken when the WL test says there is no isomorphism it is not guaranteed the GNN will map the two graphs to different embeddings,  $\psi$  is set to be non-injective by applying the element wise  $x^2$  function, which is non-injective, after the *MLP*. This is shown in Equation 7.

$$h_v^{(k)} = MLP(h_v^{(k-1)}, \sum (\{h_u^{(k-1)} : u \in \mathcal{N}(v)\}))^2 \quad (7)$$

Running the same test, when the WL test determines there is no isomorphism, the GNN does not always map the two graphs to different embeddings. This makes sense since even if the  $MLP$  maps two values to 1 and  $-1$  the squared function maps them both to 1, breaking injectivity. From the two experimental results it supports the claim that GNNs must follow requirements (a) and (b) to be as powerful as the WL test.

## 5 K-WL Test

A variant of the WL test, which has been shown to be at least as powerful as the WL test, is the k-WL test. The WL test shown earlier is also known as the 1-WL test, but higher values of  $k$  can be chosen to make a more powerful test that correctly predicts that there does not exist an isomorphism between two graphs for a wider range of graph structures. This can be shown as  $k - WL \subseteq (k + 1) - WL$   $\forall_{k \geq 1}$ .

The core idea behind the  $K$ -WL test is that instead of working with individual nodes to color, tuples of nodes are used to color. Each tuple is  $k$  long represented as  $(v_1, v_2, \dots, v_k) \in V^k$ , on startup the algorithm creates every permutation of tuples that have exactly the size of  $k$ . Instead of updating the color like original WL test by computing  $c_v^{(t+1)} = HASH(c_v^{(t)}, \{c_u^{(t)} : u \in \mathcal{N}(v)\})$  which captures the neighboring nodes, the K-WL test aggregates all tuples that that differ in one vertex of the target tuple into a multiset. The aggregated multiset is then sent through a injective  $HASH$  function to update the color assigned to the target tuple. While using higher values for  $k$  increases the expressive power of the WL test, it comes with a time complexity cost. The time complexity for the K-WL test is  $O(|V|^{k+1})$  since there are  $|V|^k$  tuples created, and for each tuple during aggregation every vertex must be checked to ensure the tuple differs by one vertex value, so  $|V|^k$  is multiplied by another  $|V|$  to get  $O(|V|^{k+1})$ .

With the time complexity of the K-WL test following to the power of  $k + 1$ , the value of  $k$  must be small or the test will not be able to run in an acceptable amount of time. This algorithm is then shown to be converted to a GNN shown in Section 6 using a similar methodology. (2)

## 6 K-GNNs

K-GNNs take inspiration from the K-WL test in order to be as powerful as the K-WL test. Since the K-WL test is at least as powerful as the WL test for all  $k \geq 1$ , K-GNNs are also at least as powerful as the WL test when  $k \geq 1$ . To do this the authors follow a similar pattern to the K-WL test which is outlined in this section.

A K-GNN first collects all  $k$  element subsets  $V^k$  over  $V$  and let  $s = \{s_1, \dots, s_k\}$  be a  $k$  sized set in  $V^k$ , then define the neighborhood of  $s$  as shown in Equation 8.

$$N(s) = \{t \in V^k : |s \cap t| = k - 1\} \quad (8)$$

This essentially maps the vertices in the graphs to different  $k$  long tuples then redefines the neighborhood of the tuple to any tuple that is different in one element. This takes the same form as the original K-WL test.  $N_L(s)$  is then denoted as the local neighborhood such that  $(v, w) \in E$  for unique  $v \in t/s$  and unique  $w \in t/s$ . This means if the two tuples have vertices that are connected in the original graph by an edge the two tuples are considered in the same  $N_L(s)$ . The global neighborhood,  $N_G(s)$ , is then defined as all elements in  $N(s)$  that are not in  $N_L(s)$  or  $N_G(s) = N(s)/N_L(s)$ . This keeps the local and global neighborhoods disjoint from each other. The message passing done uses both the global and local neighborhoods to get both a local and global view of the relationship between tuples which in turn reveals information about the original graph itself. The message passing function is shown below in Equation 9.

$$f_k^{(t)}(s) = \sigma(f_k^{(t-1)}(s) \cdot W_1^{(t)} + \sum_{u \in N_L(s) \cup N_G(s)} f_k^{(t-1)}(u) \cdot W_2^{(t)}) \quad (9)$$

201 In this message passing algorithm, the local and global neighborhoods are combined together. The  
 202 reason to keep the local and the global neighborhoods disjoint is that the authors propose separating  
 203 the local and global neighborhoods so a different weight can be applied to the aggregation of  
 204 each neighborhood separately in other works. This is at least as powerful as the K-WL test since  
 205 aggregation occurs over  $N(s)$  which is defined the same way as the K-WL test, sets differing in one  
 206 vertex value. Aggregation also occurs over the local neighborhoods of each  $k$  tuple,  $N_L(s)$ , which  
 207 helps preserve the original graph structure.

208 As well as proposing K-GNN for individual, fixed values of K, they also propose a hierarchical  
 209 variation. This variation uses different values of K, pools features from the resulting graphs then uses  
 210 the pooled features as input to a feed forward network for the final classification. This ensures local  
 211 information is used, when  $k = 1$ , as well as using higher order and neighborhood level information  
 212 from higher values of  $k$ . For the variation used by the authors,  $k = 1, 2, 3$  which is shown below in  
 213 Figure 2.

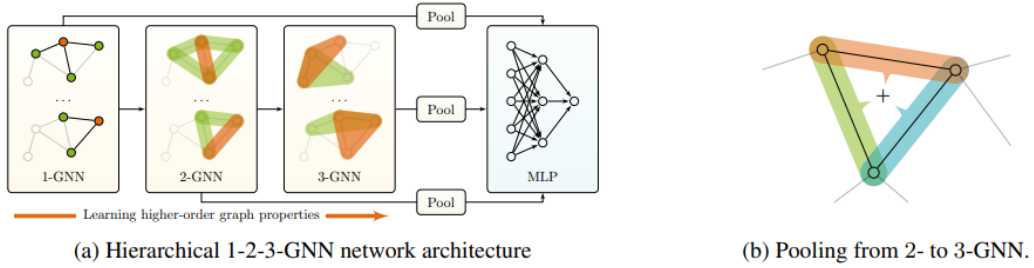


Figure 2: Hierarchical K-GNN With K = 1, 2, 3

214 They then compare the hierarchical K-GNN with a K-GNN when  $k = 1$  as well as base GNN  
 215 baselines. Their results showed that the hierarchical K-GNN outperformed the single 1-GNN, but  
 216 would only outperform baseline GNN or graph kernels, such as the WL test, on certain datasets.

217 The biggest flaw of this paper is the time complexity to run the K-GNN. This follows from the  
 218 exponential time complexity from the K-WL test with respect to  $k$ ,  $O(|V|^{k+1})$ , since the K-GNN  
 219 takes the same structure as the K-WL test. The time complexity increases even more when applying  
 220 the hierarchical model which runs K-GNN for multiple different values of  $k$ . The one positive of the  
 221 hierarchical version is that for each different value of  $k$ , each separate K-GNN can be run in parallel  
 222 while learning different information and views of the same graph.

## 223 7 Identity Aware GNNs

224 Another modification to MP-GNNs that has been shown to be more powerful than the 1-WL test  
 225 is the identity-aware graph neural network. (3) This network changes how aggregating works by  
 226 instead using the ego network of the target node instead of only the neighboring nodes. The  $K$ -hop  
 227 ego network  $\mathcal{G}_v^{(K)}$  of the target node  $v$  is extracted, which is defined as every node and edge that is at  
 228 most  $K$  hops away from node  $v$ . Then  $K$  rounds of message passing is done for each node  $u \in \mathcal{G}_v^{(K)}$   
 229 which is shown below in Equations 10 and 11.

$$m_s^{(K)} = MSG_{1[s=v]}^{(k)}(h_s^{(k-1)}) \quad (10)$$

$$h_u^{(k)} = AGG^{(k)}(\{m_s^{(k)}, s \in \mathcal{N}(u)\}, h_u^{(k-1)}) \quad (11)$$

231 Where only  $h_v^{(K)}$  is used as the embedding representation for node  $v$  after applying  $K$  rounds. This  
 232 means that even though message passing is done for every node,  $u \in \mathcal{G}_v^{(K)}$ , it is only assigned to the  
 233 target node,  $v$ . The process of finding the ego network, performing message passing, then assigning  
 234 the new feature vector to the target node  $v$  is done for all  $v \in V$ . This takes much more time to

compute compared to regular MP-GNNs since message passing needs to be done on each node’s ego network. This is shown to be worth the extra time by the accuracy gains shown on different datasets.

To validate their findings the authors augment existing MP-GNN architectures to include their proposed method by updating how the target node  $v$  is updated with their ego network approach. They modify graph convolution network (GCN), GraphSAGE, GIN, and graph attention networks (GAT). (4) (5) (1) (6) Comparing the best performing identity based augmentation with a non-augmented GNN an accuracy gain ranging between 1% to 55% depending on the dataset is observed. This shows that the extra time to compute is worth it by the large amount of possible accuracy gains.

To show adding this augmentation to a MP-GNN makes it more powerful than the 1-WL test **Proposition 1** is proposed.

**Proposition 1:** ID-GNN version of GIN can differentiate any graph that GIN can differentiate, while being able to differentiate certain graphs that GIN fails to distinguish.

To prove this they simply set  $MSG_{1[s=v]}^k$  in their implementation to the base message function which makes their implementation the same as GIN. To show identity aware GNNs can differentiate certain graphs that GIN fails to distinguish, they observe that identity aware GNNs can count cycles through the  $k$  hop ego network message passing reaching farther nodes, while GIN cannot count cycles due to only aggregating immediate neighbors. This is extended in **Proposition 2**.

**Proposition 2:** For any node  $v$ , there exists a  $K$ -layer ID-GNN instantiation that can learn an embedding  $h_v^{(K)}$  where the  $j$ -th dimension  $h_v^{(K)}[j]$  equals the number of length  $j$  cycles starting and ending at node  $v$ , for  $j = 1, \dots, K$ .

To show this the authors use induction. For  $K = 1$ , the ID-GNN can count length one paths from  $u$  to  $v$ . For  $K = n$ , the ID-GNN can count length  $n$  paths from  $u$  to  $v$ , and for  $K = n + 1$  the ID-GNN can count length  $n + 1$  paths from  $u$  to  $v$ . So as long as  $K$  is large enough, which also requires more compute time, can determine how many cycles exist of length  $j = 1, \dots, K$ .

Since GIN is as powerful as the 1-WL test and ID-GNNs are more powerful than GIN, GIN is a strict subset of ID-GNNs. This claim is experimentally backed up by the testing done on different datasets showing ID-GNNs outperform classic GNNs like GCN and GIN.

$$1 - WL = GIN \subset ID - GNN \quad (12)$$

While ID-GNNs are theoretically more powerful than both GIN and the 1-WL test, scalability becomes an issue with this GNN structure. This is mainly because of the extra message passing that must occur for every vertex  $v \in V$  on the ego network. While message passing on the ego network can be done in parallel, as well as running every ego network message passing stage, performing this can be expensive in time complexity. In the paper, the authors observe that ID-GNNs run 3.8 times slower compared to regular GNNs.

To overcome this, the authors also propose a fast version of their implementation. In this implementation, the number of cycles the node is part of is concatenated to the feature vector of that node. Then a classic MP-GNN is run on the input graph with the number of cycles appended to each feature node’s feature vector. The authors decide to add the number of cycles since this is what is observed as the main benefit given from ID-GNN performing message passing on the extracted ego networks. Since a regular MP-GNN is run on the graph, the runtime is the same as a regular MP-GNN. The fast version of the ID-GNN tended to perform worse on different tasks compared to ID-GNN, but tended to perform as well or better than its original counterparts such as GIN, GCN, and GAT. Whether or not to use the fast version comes down to a common tradeoff of speed and expressive power.

## 8 Ordered Subgraph Aggregation Networks

The authors of Ordered Subgraph Aggregation Networks (7), propose a methodology similar to K-GNNs (2) by gathering all combinations of vertices that are  $k$  length. K-GNNs gathers these in the form of a  $k$  long tuples of vertices, while in this paper the authors propose gathering vertices in ordered sub-graphs that have  $k$  vertices in each subgraph. Since the subgraphs are ordered the subgraph containing the vertices  $(v_1, v_2, v_3)$  is treated as a separate sub graph created from  $(v_2, v_1, v_3)$ .

The core idea behind their implementation,  $k$ -ordered sub-graph MP-GNNs ( $k$ -OSANs), is to encode the importance of each sub-graph to its relation to each vertex. The core aspect where  $k$ -OSANs

285 differ from regular MP-GNNs is all the covered MP-GNNs have a single feature vector per vertex,  
 286 while each vertex in a  $k$ -OSAN contains a separate feature vector for each ordered sub-graph. So  
 287 each vertex,  $v$ , will have  $|V|^k$  different feature vectors, which each represents how that vertex relates  
 288 to each ordered sub-graph with  $k$  vertices. Message passing is done as usual for each vertex but each  
 289 feature vector is only aggregated with feature vectors that represent the same sub-graph. On the last  
 290 layer, vector features within each vertex are aggregated into a single feature vector per vertex, now in  
 291 the form of MP-GNNs covered earlier. A *READOUT* function is then used on the graph to map the  
 292 graph to a vector representation. So the graph representation goes from  $|V|^k$  feature vectors for each  
 293 vertex representing the relationship between the vertex and each ordered subgraph with  $k$  vertices, to  
 294 a graph where each vertex has a single feature vector, to a single vector after applying a *READOUT*  
 295 function.

296 The initialization of the feature vectors is shown below in Equation 13.

$$h_{v,g}^{(0)} := \text{UPDATE}(\text{atp}(v, t(g))) \in \mathbb{R}^d \quad (13)$$

297 Where  $h_{v,g}^{(0)}$  represents the feature vector representing the relationship between vertex  $v$  and the  
 298 ordered subgraph  $g$ , *UPDATE* is a differentiable function such as a neural network, while  $t$  and  
 299 *atp* represent encoding functions for the ordered subgraph vertices and the vertex with the encoded  
 300 ordered subgraph respectively.

301 Another feature is concatenated to each  $h_{v,g}^{(0)}$ ,  $\pi_{v,g}$ , which is used in later aggregations to select  
 302 admissible ordered subgraphs for the vertex  $v$ . This is shown below in Equation 14.

$$\pi_{v,g} := \text{UPDATE}_\pi(\text{atp}(v, t(g))) \quad (14)$$

303 To aggregate the features representing the same subgraph of neighboring vertices Equation 15 is run  
 304 for each subgraph feature vector in every vertex on each layer.

$$h_{v,g}^{(i+1)} := \text{UPDATE}^{(i+1)}(h_{v,g}^{(i)}, \text{AGG}^{(i+1)}(\{h_{u,g}^{(i)} | u \in N_G(v)\})) \quad (15)$$

305 This takes the same form as previous aggregation functions, but only looks at neighboring features  
 306 that represent the same ordered subgraph,  $g$ . This can be run for however many layers until the final  
 307 aggregation in each vertex occurs on the last layer, *SAGG*. This aggregates all the feature vectors  
 308 within each vertex. The step also only aggregates features, which represent ordered-subgraphs  $g$ , that  
 309 are learned to have influence on the vertex,  $v$ . This is represented by the feature  $\pi_{v,g} \neq 0$ , which was  
 310 initialized and can be learned during training. This process is shown below in Equation 16.

$$h_v^{(T)} := \text{SAGG}(\{h_{v,g}^{(T)} | g \in G_k, \pi_{v,g} \neq 0\}) \quad (16)$$

311 Now that each vertex has a single feature vector, a *READOUT* function can map the graph features  
 312 to a single vector representation for the specified task. Before covering theoretical claims relating to  
 313 the  $k$ -OSANs, the ordered subgraph-WL test is covered.

314 The ordered subgraph-WL test inspired  $k$ -OSANs, so the structure for both are similar. The ordered  
 315 subgraph-WL test is initialized the same way except  $\pi_{v,g}$  is not initialized, initial colors are assigned  
 316 for every subgraph of length  $k$  in every vertex. The initial color for each subgraph,  $g$ , and vertex,  $v$ ,  
 317 pair is initialized as  $C_0(v, g) := \text{atp}(v, t(g))$ . Then for each iteration the color is update to reflect  
 318 Equation 17.

$$C_{i+1}(v, g) := \text{RELABEL}((C_i(v, g), \{C_i(u, g) | u \in N_G(v)\})) \quad (17)$$

319 Then to assign a single color to each node, or in other words aggregate the colors that represent each  
 320 subgraph within each node Equation 18 is run for each vertex.

$$C(v) := \text{RELABEL}(\{C_\infty(v, g) | g \in G_k\}) \quad (18)$$

321 Then the regular WL test is run comparing the multiset of colors between each graph to determine if  
 322 there does not exist an isomorphism between two graphs.

First to show that  $k$ -OSAN  $\subset (k+1)$ -OSAN,  $k$ -OSAN can be simulated by  $(k+1)$ -OSAN by simply mapping each subgraph,  $g = (v_1, v_2, \dots, v_k)$  to  $g = (v_1, v_2, \dots, v_k, v_{k+1})$  where  $v_{k+1}$  is a dummy vertex that is the same for each subgraph. This shows that the mapping exists from  $k$ -OSAN to  $(k+1)$ -OSAN. This map does not exist from  $(k+1)$ -OSAN to  $k$ -OSAN since there is no way to map  $g = (v_1, v_2, \dots, v_k, v_{k+1})$  to  $g = (v_1, v_2, \dots, v_k)$  since there is no open position to map  $v_{k+1}$  to. Therefore,  $k$ -OSAN  $\subset (k+1)$ -OSAN.

Next it is shown that  $k$ -OSAN is more expressive than 1-WL for  $k \geq 1$ . This can be shown by simulating 1-WL with 1-OSAN. This means that in 1-OSAN, every vertex contains a feature vector representing every vertex as its own subgraph. Then set  $\pi_{v,g} = 1$  if and only if  $v = g$ , otherwise  $\pi_{v,g} = 0$ . So only the feature vector representing its own node is used in the *SAGG* step shown in Equation 16. This means only the feature vectors representing the vertex itself make it through to the last message passing round, which can be thought of as the color assignment for each vertex in the 1-WL test. Then iterations of message passing can be done to emulate aggregating the color assignment of each neighboring vertex to finish emulating the 1-WL test. Since this mapping works for  $k = 1$  and  $k$ -OSAN  $\subset (k+1)$ -OSAN, this also holds for  $k \geq 1$ . Therefore, for  $k \geq 1$ ,  $k$ -OSAN is more expressive than the 1-WL test.

These two proofs show that even for a small value of  $k$ ,  $k$ -OSAN can be more expressive than the 1-WL test as well as get more expressive while increasing  $k$ .

This variation of a MP-GNN has a main problem of time complexity compared to GIN. Since each vertex contains a feature vector representing each subgraph,  $g$ , there are a total of  $|V| \cdot |V|^k$  feature vectors. Since each of those feature vectors need to be processed, the message passing required scales exponentially with  $k$ . Even though this can be done in parallel, the number of parallel computations to do still grows exponentially with a larger  $k$  value so the time to train this type of network would take much longer compared to a regular MP-GNN such as GIN. (1)

## 9 Path-WL

In the paper, The Expressive Power of Path-Based Graph Neural Networks (8), the authors first propose a variation of the 1-WL test that is strictly more expressive called Path-WL. They then adapt this to a MP-GNN which in turn is also strictly more powerful than the 1-WL test. The core idea of the path-WL test is to aggregate nodes that are farther than one hop away as well as encode the distance of each of the nodes. As a reminder to the 1-WL test, it only aggregates node colors that are one hop away and updates the color of vertex  $v$  on iteration  $i$  shown in Equation 19.

$$c_v^{(i)} = \text{HASH}(c_v^{(i-1)}, \{\{c_u^{(i-1)} | u \in \mathcal{N}(v)\}\}) \quad (19)$$

This is extended by path-WL by using  $d - \mathcal{P}_v^l$ , which represents the set of all paths of length up to  $l$  starting at node  $v$ . This can be defined in Equation 20.

$$d - \mathcal{P}_v^l := \{\{(c_v, \eta_{vv}^d), (c_{v_1}, \eta_{vv_1}^d), \dots, (c_{v_k}, \eta_{vv_k}^d)\} | (v, v_1, \dots, v_k) \in \mathcal{P}_v^l, k \leq l\} \quad (20)$$

Where  $c_v$  is the color of node  $v$  and  $\eta_{vv_i}^d$  is the shortest path distance from  $v$  to  $v_i$  if the shortest path distance is less than or equal to  $d$  and  $\emptyset$  otherwise. This means that if  $d = 0$ , the shortest distance encoding from  $v$  to  $v_i$ ,  $\eta_{vv_i}^0$ , will not be included since it is set to the empty set, unless  $v = v_i$ . In other words,  $d - \mathcal{P}_v^l$  represents the colors along the shortest path to nodes that are at most a distance of  $l$  hops away from  $v$  and encodes the shortest distance to the node  $v_i$  from  $v$  if it is at most  $d$  hops away from  $v$ . This is then used to update the color assigned to the node  $v$  on iteration  $i$  by Equation 21.

$$c_v^{(i)} = \text{HASH}(d - \mathcal{P}_v^{l, (i-1)}) \quad (21)$$

This is done for every  $v \in V$  for any number of iterations. The authors then explain that the path-WL test scales with larger  $d$  and  $l$  values in expressiveness. They claim that for every  $d' \geq d \geq 0$  and  $l' \geq l \geq 1$  it holds that  $d - \text{Path-WL}^l \subseteq d' - \text{Path-WL}^l$  and  $d - \text{Path-WL}^l \subseteq d - \text{Path-WL}^{l'}$ . To prove this the authors show by induction that distance encodings,  $\eta_{vv_i}^d$  encodes the distance of farther nodes

with  $d + 1$  compared to  $d$ . The same reasoning is set for using  $l + 1$  over  $l$ , farther nodes are visited and more colors from those nodes,  $c_{v_k}$  are used when updating  $c_v$ . This gives the encoding of  $c_v$  more information about the surrounding paths which supports the author's claim.

**Proposition 4.1:** For every  $l > 1$ , every  $d \geq 0$ ,  $d$ -Path-WL<sup>*l*</sup> is strictly more expressive than 1-WL.

To show this the full proof can be found in the paper (8). The proof starts by showing that 0-Path-WL is at least as powerful as the 1-WL test. This can be done multiple ways but the simplest is to set  $l = 1$  and  $d = 0$  which makes the aggregations of Path-WL and 1-WL the same since node colors are aggregated the same way with no distance information and only nodes one hop away. To show that Path-WL is more expressive than 1-WL, they use an example that Path-WL can distinguish as not having an isomorphism between the two graphs, while 1-WL cannot determine if there does not exist an isomorphism between the two graphs. The two graphs are shown below in Figure 3. This is shown experimentally too in Section 9.1.

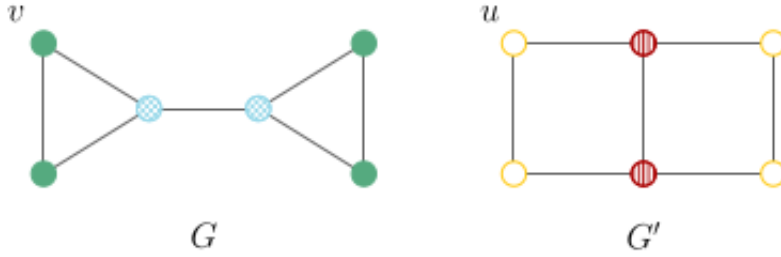


Figure 3: Two Graphs Path-WL can Distinguish While 1-WL Cannot

Since the proof shows that 0-Path-WL<sup>1</sup> is at least as powerful as the 1-WL test and there exists an example where 0-Path-WL<sup>1</sup> distinguishes a graph while 1-WL cannot,  $1\text{-WL} \subset 0\text{-Path-WL}^1$ . Since  $d\text{-Path-WL}^l \subseteq (d + 1)\text{-Path-WL}^l$  and  $d\text{-Path-WL}^l \subseteq d\text{-Path-WL}^{(l+1)}$ , shown before,  $1\text{-WL} \subset d\text{-Path-WL}^l$  for  $l > 1$  and  $d \geq 0$ .

A major downside of this paper, as well as others reviewed, is the feasibility on larger graphs. Since each vertex  $v$  explores every path at most  $l$  hops away if a graph is fully connected the number of paths blows up exponentially. For example a fully connected graph with a max degree of  $n$  will have  $O(n^l)$  paths that has a length of  $l$ .

## 9.1 Home Experimentation

For the experiment for Path-WL, the two graphs shown in Figure 3,  $G$  and  $G'$ , are tested with 0-Path-WL<sup>1</sup> and 1-WL. There does not exist an isomorphism between the two graphs so both algorithms should output there does not exist an isomorphism.

Starting with observations, both  $G$  and  $G'$  have 2 degree 3 and 4 degree 2 nodes. This means that after the first iteration of the 1-WL test the multiset of colors of  $G$  and  $G'$  will be exactly the same since each node starts with the same color assigned and only neighbors are aggregated. Since the multisets are the same, if only one iteration is done the 1-WL test could not distinguish between  $G$  and  $G'$ . On the other hand, 0-Path-WL can encode the colors along the shortest path to each other node that is at most  $l$  hops away. Take the most upper left node in  $G$  to the upper right node in  $G$ , the shortest distance is 3, while the same two nodes in  $G'$  are 2 hops away from each other. Information such as this is encoded from 0-Path-WL while not observed while running 1-WL.

When running the two tests on these two graphs, the 0-Path-WL test is able to distinguish the two graphs as not having an isomorphism, while 1-WL is not able to distinguish the two graphs after 20 iterations. This follows the findings by the authors and backs up the initial observations of the two graphs and the predictions that were made on how the two algorithms would perform, since if the 1-WL test cannot distinguish two graphs after 20 iterations it cannot distinguish the two graphs after only 1 iteration either, since 1-WL checks the multisets of colors after each iteration.

## 10 Paper Comparisons

All of the proposed MP-GNN variants covered in this paper are at least as powerful as the 1-WL test. K-GNNs, identity aware GNNs, ordered subgraph aggregation networks, and path-wl are all more powerful than the 1-WL test. This is caused by those MP-GNN variants to be modeled off a more expressive variation of WL test than the 1-WL test, such as K-GNNs modeling the  $k$ -WL test. A downside of these more powerful algorithms is that they all have a runtime that is much slower than GIN, which is as powerful as the 1-WL test. GIN runs in polynomial time and can run on large graphs since it runs a standard message passing algorithm that only takes into account neighboring nodes. Variants that look at different partitions of the graph such as K-GNN and ordered subgraph aggregation networks grow exponentially when choosing larger values of  $k$ . This is why in an experimental setup small values for  $k$  are used, such as a max of  $k = 3$  for K-GNNs. Path-WL and identity aware GNNs relay on the vertices that are a certain distance away from the target vertex,  $v$ , while aggregating. So in the worst case, fully connected graph, the time complexity would increase exponentially with a factor of the path length to explore. This is also seen in experiments in ID-GNNs whose authors claim that the running time is 3.8 times longer than a normal MP-GNN, such as GCN or GAT. (4)(6) These two problems do not occur in the original GIN implementation since only the base graph is used, so no partitions of vertices are made, and when aggregation occurs, the target vertex,  $v$ , only aggregates vertices that are neighbors, so one hop away. This brings up the a tradeoff of expressiveness and time complexity. GIN is the least expressive of all the covered MP-GNNs in this paper but runs the fastest compared to the other covered GNN variants.

## 11 Conclusion

In conclusion, the initial findings in the Graph Isomorphism Network (1) paper set a groundwork for the limitations of MP-GNNs compared to the 1-WL test. This sets a good theoretical benchmark for MP-GNN architectures to overcome. Other research has been done to attempt to theoretically overcome this barrier, becoming more expressive than the 1-WL test, by modifying the existing MP-GNN architectures. (2) (3) (7) (8) By overcoming the the 1-WL test, these different MP-GNN architectures have also experimentally been shown to outperform GIN, since GIN is as expressive as the 1-WL test. Home experimentation has also been done to experimentally back up the theoretical results in some papers such as showing GIN can only be as powerful as the 1-WL test when certain conditions are met such as injectivity.

While many different ways have been shown to overcome the 1-WL test, as well as creating GNNs that are as powerful as the K-WL test, more can be done to overcome this limitation. Some straightforward next directions would be to combine different strategies to make MP-GNNs more expressive into one. A major pitfall to most of the algorithms that overcome the 1-WL limitation is that they take much time to process. Original message passing is inherently parallel, computing each vertex in parallel, which raises the question of scalability and how to parallelize each algorithm. Some of the algorithms, such as Identity Aware GNNs (3) and Path-WL (8), perform more operations for each vertex which can still be made parallel. In the case of Identity Aware GNNs, a round of message passing is done for each ego network of each vertex which can be made parallel. In Path-WL,  $d - \mathcal{P}_v^l$  can be calculated for each vertex  $v \in V$  in parallel. Other algorithms such as K-GNNs and the ordered subgraph approach explode exponentially with a higher  $k$  value which makes them harder to efficiently process.

With parallel programs in the mix, it is also important to be able to train all of these network structures in a distributed cluster of GPUs which has been shown to be done with classic MP-GNNs, such as GIN, but not expanded to more theoretical based GNNs. In summary, while MP-GNNs can theoretically overcome the 1-WL test it needs to be shown that these models can be scaled up to large graphs through parallel programs and distributed training.

## References

- [1] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" *arXiv preprint arXiv:1810.00826*, 2018.

- 455 [2] C. Morris, M. Ritzert, M. Fey, W. L. Hamilton, J. E. Lenssen, G. Rattan, and M. Grohe,  
456 “Weisfeiler and leman go neural: Higher-order graph neural networks,” in *Proceedings of the*  
457 *AAAI conference on artificial intelligence*, vol. 33, no. 01, 2019, pp. 4602–4609.
- 458 [3] J. You, J. M. Gomes-Selman, R. Ying, and J. Leskovec, “Identity-aware graph neural networks,”  
459 in *Proceedings of the AAAI conference on artificial intelligence*, vol. 35, no. 12, 2021, pp.  
460 10 737–10 745.
- 461 [4] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,”  
462 2017. [Online]. Available: <https://arxiv.org/abs/1609.02907>
- 463 [5] W. L. Hamilton, R. Ying, and J. Leskovec, “Inductive representation learning on large graphs,”  
464 2018. [Online]. Available: <https://arxiv.org/abs/1706.02216>
- 465 [6] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph attention  
466 networks,” 2018. [Online]. Available: <https://arxiv.org/abs/1710.10903>
- 467 [7] C. Qian, G. Rattan, F. Geerts, M. Niepert, and C. Morris, “Ordered subgraph aggregation  
468 networks,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 21 030–21 045,  
469 2022.
- 470 [8] C. Graziani, T. Drucks, F. Jogl, M. Bianchini, F. Scarselli, T. Gartner *et al.*, “The expressive power  
471 of path-based graph neural networks,” *PROCEEDINGS OF MACHINE LEARNING RESEARCH*,  
472 vol. 235, pp. 16 226–16 249, 2024.